

JobIntel Agent

Evidence-Grounded AI Job Analysis Agent

项目设计与 AI 协作开发规格说明书

文档项	内容
版本	v1.0
文档用途	正式设计阅读 / 分阶段开发 / 与 AI 协作编码 / 简历项目落地
项目目标	将现有 Sales Research Agent 工程骨架改造为可验证的求职岗位分析 Agent
V1 核心范围	岗位解析、候选人证据检索、匹配判断、确定性评分、证据校验、结果持久化、Evaluation
明确不做	自动海投、多智能体、复杂 RAG、真实招聘网站爬虫、Kubernetes 等非核心扩展

核心定位：不是“求职聊天机器人”，而是一个可验证、可评测、具备工具自主选择能力的求职决策 Agent。

目录

1. 项目概述与目标
 2. 设计原则与范围控制
 3. 总体系统架构
 4. 原项目改造策略
 5. 核心业务流程
 6. 数据模型设计
 7. MCP Tool 设计
 8. Agent Core 与 Prompt 设计
 9. Evidence Guardrail
 10. 匹配与评分机制
 11. 数据库与 Repository
 12. Evaluation 体系
 13. 测试策略
 14. 开发里程碑
 15. AI 协作开发规范
 16. 简历与面试展示重点
 17. V2 扩展路线
- 附录 A. 推荐目录结构
- 附录 B. AI 任务指令模板
- 附录 C. Definition of Done

1. 项目概述与目标

1.1 背景

目标是从一个已经具有 Agent Tool Loop、MCP Server、Pydantic Structured Output、Provider Abstraction 与 Provenance Guardrail 的小型实战项目出发，保留其工程骨架，将销售研究领域替换为求职岗位分析领域，并补充更强的证据匹配、确定性评分与 Evaluation。

1.2 用户问题

V1 只解决一个高价值且边界清晰的问题：给定岗位 JD 与候选人履历，系统判断“是否值得申请”，并说明为什么。

输入：Job Description / Job ID + Candidate Profile

输出：JobAnalysis（匹配度、申请建议、逐条要求匹配、证据、技能缺口、简历建议、面试准备项）

1.3 成功标准

维度	V1 成功标准
Agent 行为	能够根据上下文自主决定是否调用 get_job / get_candidate_profile / search_candidate_evidence / save_analysis 等工具
可信性	任何“候选人具备某能力”的结论必须绑定 evidence_id；无证据时不得标记为 matched
评分	最终分数由程序根据结构化匹配结果计算，LLM 不直接输出主观总分
可评测性	具备可重复运行的离线 eval 数据集与核心指标
工程性	模型、工具、Repository、业务服务、Guardrail、Evaluation 解耦；可测试、可替换 Provider
可展示性	README 能说明架构、Agent 决策过程、Guardrail、指标和 Demo

2. 设计原则与范围控制

证据优先

模型可以做语义判断，但不能凭印象声称候选人会某项技术。所有正向匹配必须能追溯到候选人真实证据。

工具优先于超长上下文

Agent 在需要信息时主动调用 Tool，而不是把全部数据一次性塞进 Prompt。

LLM 做语义，代码做规则

LLM 负责抽取和语义匹配；程序负责评分、合法性校验、状态持久化和硬约束。

小而完整

优先完成一个闭环，不用技术栈堆砌。

可评测

每一个重要 Agent 能力都尽量对应可量化指标。

先离线后联网

V1 使用 SQLite fixture / 手工输入 JD；联网抓取岗位是 V2。

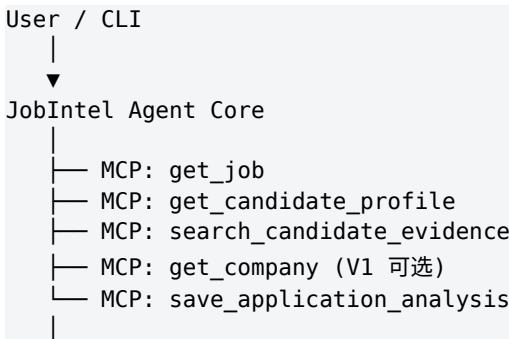
2.1 V1 范围

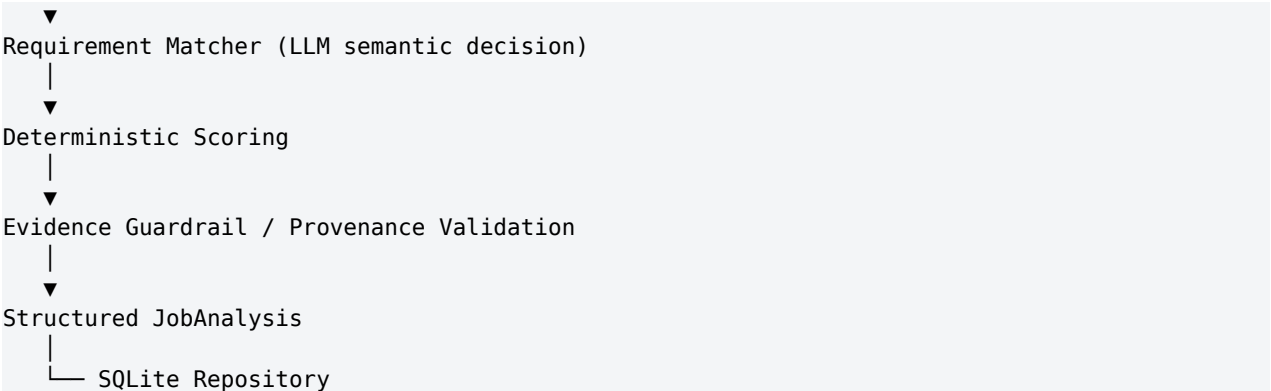
- 岗位 JD 结构化解析
- 候选人结构化 Profile
- 岗位要求分类：must / preferred / bonus
- 候选人 Evidence 检索
- RequirementMatch 生成
- 确定性 Match Score
- Provenance / Unsupported Claim Guardrail
- 结果写入 SQLite
- 离线 Evaluation
- CLI 或简单 Web 入口（二选一，CLI 优先）

2.2 V1 明确排除

- 自动海投和自动提交表单
- 多 Agent 协作
- LangGraph/CrewAI 等额外编排框架
- 复杂向量数据库和 RAG Pipeline
- Redis/Kafka/Kubernetes
- LinkedIn/BOSS 等网站爬虫
- 自动生成整份新简历
- Email Agent / Calendar Agent

3. 总体系统架构





3.1 责任边界

模块	主要职责	不负责
Agent Core	对话状态、调用 LLM、执行 Tool Loop、决定何时结束	数据库细节、评分公式
MCP Server	向 Agent 暴露稳定的业务工具接口	Agent 决策
Repository	SQLite CRUD、领域对象装配	LLM Prompt
JD Parser	将岗位描述转为结构化 Requirements	候选人匹配
Evidence Search	从 CandidateProfile 中查找潜在支持证据	最终评分
Matcher	判断 requirement 与 evidence 的语义关系	持久化
Scoring	根据 match status + importance 计算分数	语义推理
Guardrail	验证 evidence_id 与 claims 的可追溯性	重新生成业务内容
Evaluation	批量运行测试案例、统计指标	线上业务请求

4. 原项目改造策略

原则：不要“重写一个新项目”，而是先复现并理解原 Agent 骨架，再逐层替换领域模型和 Repository。核心目标是能够清楚解释哪些能力来自原骨架、哪些是自己的业务与可靠性改造。

原模块/概念	JobIntel 处理	修改程度	重点
providers/	保留 Provider abstraction	低	理解 run_turn / structured output / tool call 适配
agent/core.py	保留 Tool Loop 主体	中低	先读懂再改，不在第一阶段重构
agent/tools.py	改为 JobIntel tools	高	Tool schema 与描述直接影响 Agent 选择
agent/prompts.py	全部重写	高	约束 evidence-grounded 行为

models.py	全部替换为 Job domain	高	这是系统契约核心
CRM Repository	替换为 JobRepository	高	数据表与领域装配
MCP Server	保留 FastMCP 结构，重写工具	中高	读写工具区分
provenance guardrail	思想保留、逻辑增强	高	从 CRM ID 校验升级为 Evidence Claim 校验
evaluation	新增	新增	个人原创加分重点

4.1 必须删除的销售领域概念

- Contact / Seniority
- Opportunity / OpportunityStage
- OutreachDraft
- Buying Signal （V1 不需要；V2 可重构为 CompanySignal）

5. 核心业务流程

5.1 主流程

1. 用户提供 job_id 或直接提交 JD。
2. Agent 获取岗位完整信息。
3. 系统解析岗位要求并区分 must / preferred / bonus。
4. Agent 获取候选人结构化 Profile。
5. 针对重要 Requirement，Agent 调用 search_candidate_evidence。
6. Matcher 将 requirement 与 evidence 判断为 matched / partial / missing。
7. 程序根据 RequirementMatch 计算总分。
8. Guardrail 验证所有正向结论引用的 evidence_id 均真实存在且来自工具结果。
9. 生成结构化 JobAnalysis。
10. 用户确认后或按 V1 默认策略写入 analysis 记录。

5.2 Agent 必须体现“自主性”的位置

不要把所有调用写成固定 Pipeline。Agent 至少要在以下位置表现工具选择能力：

- 岗位信息不足时决定调用 get_job。
- 需要候选人履历时调用 get_candidate_profile。
- 对某项要求没有足够依据时调用 search_candidate_evidence。
- 可选：需要公司背景时调用 get_company。
- 完成分析后调用 save_application_analysis。

注意：可以规定完成任务的最低工具集合，但不要把每一步都写死成固定函数顺序，否则项目更像 Pipeline 而不是 Agent。

6. 数据模型设计

6.1 JobPosting

```
class JobPosting(BaseModel):
    job_id: str
    company: str
    title: str
    location: str | None
    employment_type: str | None
    description: str
    requirements: list[JobRequirement]
    source_url: str | None
```

6.2 JobRequirement

```
class JobRequirement(BaseModel):
    requirement_id: str
    text: str
    category: Literal["skill", "experience", "education", "project", "language", "other"]
    importance: Literal["must", "preferred", "bonus"]
    normalized_skill: str | None
```

6.3 CandidateProfile 与 CandidateEvidence

```
class CandidateEvidence(BaseModel):
    evidence_id: str
    evidence_type: Literal["education", "experience", "project", "skill", "certificate",
"other"]
    title: str
    content: str
    skills: list[str] = []

class CandidateProfile(BaseModel):
    candidate_id: str
    summary: str | None
    evidence: list[CandidateEvidence]
```

6.4 RequirementMatch

```
class RequirementMatch(BaseModel):
    requirement_id: str
    status: Literal["matched", "partial", "missing"]
    evidence_ids: list[str]
    confidence: float
    reason: str
```

6.5 JobAnalysis

```
class JobAnalysis(BaseModel):
    job_id: str
    score: int
    recommendation: Literal["strong_apply", "apply", "low_priority", "skip"]
    requirement_matches: list[RequirementMatch]
    strengths: list[str]
    missing_skills: list[str]
```

```
resume_suggestions: list[str]
interview_topics: list[str]
next_action: str
saved_record_id: str | None = None
```

6.6 模型设计约束

- 跨模块传递尽量使用 Pydantic Model，不使用语义不明确的 dict。
- 所有可追溯实体必须有稳定 ID。
- evidence_id 一旦生成，在同一候选人 Profile 版本内必须稳定。
- JobAnalysis 中的正向技能结论必须能够映射到 RequirementMatch.evidence_ids。

7. MCP Tool 设计

7.1 V1 Tool 集合

Tool	类型	用途	V1 数据源
get_job(job_id)	Read	获取完整岗位与 Requirements	SQLite / fixture
parse_job_requirements(job_id or jd_text)	Read/Compute	从 JD 抽取并标准化岗位要求	LLM + parser service
get_candidate_profile(candidate_id)	Read	获取候选人结构化履历	SQLite / JSON
search_candidate_evidence(query, top_k)	Read	检索支持某项岗位要求的候选人证据	结构化检索 + 可选语义匹配
get_company(company_id)	Read, 可选	读取公司基本信息	SQLite
save_application_analysis(payload)	Write	保存分析结果	SQLite

7.2 Tool 设计规则

- Tool 名称必须描述动作，不使用模糊名称。
- Tool Description 应说明“什么时候调用”和“什么时候不要调用”。
- Read Tool 不产生副作用；Write Tool 必须显式区分。
- Tool 输出结构化、稳定、带 ID。
- 错误使用领域化错误码，例如 JOB_NOT_FOUND / INVALID_EVIDENCE_ID。
- 不要让一个 Tool 同时承担“查询 + 推理 + 写入”三个职责。

7.3 核心 Tool : search_candidate_evidence

Input:

```
query: str
top_k: int = 5
```

Output:


```
list[CandidateEvidenceHit]

CandidateEvidenceHit:
  evidence_id
  title
  content
  evidence_type
  lexical_score / semantic_score (可选)
```

V1 不要求向量数据库。可以采用 normalized skill exact match + keyword/fuzzy match，再由 LLM 对候选证据进行语义判断。只有当数据规模扩大到多份简历、作品集、GitHub 文档后，才引入 embedding / vector search。

8. Agent Core 与 Prompt 设计

8.1 Agent Loop

```
messages
  ↓
provider.run_turn()
  ↓
存在 tool_calls ? 一否——→ structured final output
  | 是
  ↓
execute tool(s)
  ↓
append tool result
  ↓
下一轮 LLM
  ↓
Guardrail → accept / reject / repair
```

8.2 Prompt 最低约束

You are JobIntel, an evidence-grounded job analysis agent.

You MUST:

1. Retrieve sufficient job information before analysis.
2. Inspect the candidate profile.
3. Search candidate evidence for important requirements.
4. Never mark a requirement as matched without evidence IDs.
5. Distinguish must-have, preferred, and bonus requirements.
6. Use tools when information is missing.
7. Return only the required structured JobAnalysis schema.

8.3 Prompt 中不要做的事

- 不要让模型直接“凭感觉打 0-100 分”。
- 不要把所有业务规则只写在 Prompt 中；关键约束必须由代码校验。
- 不要把完整候选人所有原始材料都塞进 System Prompt。
- 不要要求模型自己“记住”哪些 evidence 是真实工具返回的；在程序侧记录 provenance。

9. Evidence Guardrail

9.1 核心规则

JobIntel 的核心差异化设计：任何正向能力声明必须可追溯到真实 CandidateEvidence。Prompt 只负责引导，Guardrail 负责强制执行。

```
Claim: "Candidate has production FastAPI experience"
Evidence IDs: ["P003"]
```

```
Guardrail checks:
1. P003 是否出现在本次或允许范围内的 Tool Result?
2. P003 是否属于当前 candidate_id / profile_version?
3. P003 是否真实支持对应 requirement?
4. 若证据为空, status 不得为 matched。
```

9.2 推荐校验规则

规则	触发条件	处理
Unknown Evidence	引用不存在或未返回的 evidence_id	拒绝输出，要求模型修复
Matched Without Evidence	status=matched 且 evidence_ids=[]	拒绝
Cross-Candidate Evidence	证据不属于当前候选人	拒绝
Unsupported Strength	strengths 中出现技能但无对应 matched requirement/evidence	删除或重试
Score Mismatch	LLM 输出 score 与程序计算 score 不一致	覆盖为程序结果
Invalid Recommendation	recommendation 与分数阈值不一致	程序纠正

9.3 Repair 策略

- 优先将校验错误返回给同一 Provider 进行一次 structured repair。
- 超过最大修复次数后返回可诊断错误，不无限循环。
- 日志中记录 validation_error_type，但不记录不必要的隐私原文。

10. 匹配与评分机制

10.1 语义匹配与评分分离

LLM 只输出每个 Requirement 的 matched / partial / missing 及证据和理由；最终总分由程序计算。

10.2 推荐 V1 公式

```
status_score:
  matched = 1.0
  partial = 0.5
  missing = 0.0

importance_weight:
```

```
must = 0.60
preferred = 0.25
bonus = 0.15
```

```
TotalScore = 100 * Σ(group_weight * mean(status_score in group))
```

10.3 推荐阈值

分数	Recommendation	含义
80-100	strong_apply	匹配度高，优先申请
65-79	apply	值得申请，但存在可补足缺口
45-64	low_priority	可作为备选或先补关键技能
0-44	skip	关键 must-have 缺失较多

硬门槛扩展：后续可增加“关键 must-have 缺失直接限制最高 recommendation”等规则，但 V1 不宜过度复杂。

11. 数据库与 Repository

11.1 SQLite 表

表	关键字段	用途
jobs	job_id, company_id, title, description, location, source_url	岗位原始信息
requirements	requirement_id, job_id, text, category, importance, normalized_skill	结构化岗位要求
candidate_profiles	candidate_id, profile_version, summary	候选人 Profile 版本
candidate_evidence	evidence_id, candidate_id, type, title, content, skills	可引用证据
companies	company_id, name, industry, website, description	公司基础信息
application_analysis	analysis_id, job_id, candidate_id, score, recommendation, created_at	分析历史
requirement_matches	analysis_id, requirement_id, status, evidence_ids, confidence, reason	分析明细

11.2 Repository 接口

```
class JobRepository:
    get_job(job_id)
```

```
save_job(job)
get_requirements(job_id)
get_candidate_profile(candidate_id)
search_candidate_evidence(candidate_id, query, top_k)
save_analysis(analysis)
get_analysis(analysis_id)
```

12. Evaluation 体系

这是本项目最值得新增的原创模块之一。不要只展示“Agent 能跑”，要展示“Agent 在一组固定任务上表现如何”。

12.1 Eval Case 结构

```
case_id: agent_backend_001
job: ...
candidate_profile: ...
expected:
  required_skills: [Python, FastAPI, MCP]
  expected_missing: [Redis]
  expected_matches:
    R001: matched
    R002: partial
    R003: missing
score_range: [68, 78]
```

12.2 核心指标

Metric	定义	目标方向
Requirement Extraction Recall	关键岗位要求是否抽取完整	越高越好
Importance Classification Accuracy	must/preferred/bonus 分类正确率	越高越好
Match Accuracy	matched/partial/missing 与标注一致率	越高越好
Evidence Precision	引用 evidence 是否真的支持对应结论	越高越好
Unsupported Claim Rate	无证据正向 claim / 总正向 claim	越低越好，目标接近 0
Tool Call Success Rate	正确调用并成功返回的 Tool 比例	越高越好
Tool Selection Accuracy	任务是否调用了必要工具且避免无关工具	越高越好
Latency	单案例端到端时间	可控
Token / Cost	单案例模型使用量与成本	可控

12.3 最小 Eval 规模

- 开发早期：10 个高质量案例。
- V1 发布：30-50 个案例。
- 案例要覆盖：明显匹配、明显不匹配、同义词技能、经验年限、教育要求、模糊 JD、证据冲突。

13. 测试策略

层级	测试内容	是否需要真实 LLM
Unit	Scoring、ID 校验、Repository CRUD、Model validation	否
Tool	每个 MCP Tool 正常路径 / Not Found / Invalid Input	否
Guardrail	伪造 evidence_id、空 evidence matched、跨候选人引用	否
Provider Contract	OpenAI / Anthropic adapter 输入输出契约	可 Mock
Agent Offline	FakeProvider 预设 tool call 序列，验证 Tool Loop	否
Agent Integration	少量真实模型运行，检查端到端 structured output	是
Evaluation	固定案例批量运行并输出报告	部分

13.1 必须拥有的回归用例

- 模型声称会 Redis，但 Profile 无 Redis → 必须拒绝或 missing。
- 模型引用不存在的 P999 → Guardrail 必须拦截。
- 同义表达 “REST API backend” 与 “FastAPI 服务” → 应允许 partial/matched（依据标注）。
- must-have 大量缺失时，不得给 strong_apply。
- save_application_analysis 必须保存最终程序计算分数，而非模型原始分数。

14. 开发里程碑

阶段	目标	主要产物	完成判定
M0 复现原项目	完全跑通原 Sales Agent	环境、seed、离线测试、至少一种 Provider	能够解释 Agent Loop 与 MCP 调用路径
M1 Domain Model	替换销售领域模型	JobPosting / Requirement / CandidateProfile / Evidence / JobAnalysis	模型测试通过
M2 Repository	建立 JobIntel SQLite	schema、seed data、JobRepository	CRUD 与 fixture 测试通过
M3 MCP Tools	实现核心 Read/Write Tools	get_job、profile、evidence、save	MCP Inspector / tool tests 通过
M4 Agent 行为	重写 Prompt 并接通 Tool Loop	端到端 JobAnalysis	FakeProvider + 少量真实模型通过
M5 Guardrail + Scoring	实现核心可靠性机制	validator、repair、scoring	关键负例全部拦截

M6 Evaluation	建立 30-50 case eval	runner、metrics、report	README 有真实指标表
M7 展示	整理 Demo 与文档	README、architecture、示例输出	可以用于简历/面试

14.1 开发优先级

最高优先级: M0 → M1 → M2 → M3 → M4 → M5 → M6
可推迟: 公司研究、Web URL 抓取、Browser Agent、RAG、GUI

15. AI 协作开发规范

本章用于后续将文档直接交给 AI 编码助手。原则是：一次只让 AI 完成一个边界明确、可测试的小任务，并要求它尊重已有契约。

15.1 每次给 AI 的最小上下文

- 当前任务对应的章节号。
- 相关文件内容，而不是整个仓库。
- 明确“允许修改文件”和“禁止修改文件”。
- 输入/输出 Pydantic Schema。
- 现有测试与新增验收测试。
- 是否允许新增依赖。

15.2 AI 修改代码的强制规则

- 不得未经说明重构 Agent Core。
- 不得为了方便绕过 Pydantic 类型。
- 不得将 Guardrail 规则仅移动到 Prompt。
- 不得让 LLM 替代 deterministic scoring。
- 不得新增大框架解决小问题。
- 每次代码修改必须同步给出或更新测试。
- 修改公共接口时必须列出受影响调用点。

15.3 推荐协作节奏

设计文档章节 → AI 生成实现计划 → 你审查计划 → AI 修改 1-3 个文件 → 运行测试 → 你提供报错/结果 → AI 修复 → 更新文档/README

15.4 任务拆分原则

“实现 JobIntel” 不是一个合格的 AI 任务；“按第 6.2 节新增 JobRequirement 模型并为 validation 编写 6 个单元测试” 才是。

16. 简历与面试展示重点

16.1 项目真正要讲的三点

Evidence-grounded Agent

所有匹配结论绑定真实 Evidence ID，并由代码级 provenance guardrail 阻止 unsupported claims。

MCP + Autonomous Tool Selection

Agent 根据缺失信息自主选择岗位、候选人证据、持久化等 MCP 工具，而不是固定脚本 Pipeline。

Evaluation + Deterministic Scoring

模型负责语义判断，程序负责评分；通过固定 eval cases 量化 Requirement Recall、Match Accuracy、Evidence Precision 与 Unsupported Claim Rate。

16.2 项目描述模板

建议在真正跑出指标后再填数字：

```
JobIntel - Evidence-Grounded AI Job Analysis Agent
- 基于 Python、MCP 与自定义 Agent Tool Loop 构建岗位分析 Agent，自主调用岗位、候选人证据与持久化工具完成多步骤求职决策。
- 设计 Evidence Provenance Guardrail，为每项岗位匹配结论绑定真实 Resume Evidence ID，并在代码层阻止无证据能力声明。
- 将 LLM 语义匹配与确定性评分解耦，构建离线 Evaluation Harness，评估 requirement recall、match accuracy、evidence precision、tool selection 与 unsupported claim rate。
```

17. V2 扩展路线

优先级	扩展	什么时候做	原因
P1	JD URL Fetcher	V1 完成后	让输入更真实，但不引入 Browser complexity
P1	Company Research Tool	有基本联网能力后	补充公司背景与招聘信号
P2	Resume Parser (PDF/DOCX)	结构化 Profile 稳定后	提升用户体验
P2	Vector / Hybrid Evidence Search	候选人材料规模显著扩大后	此时 RAG 才有合理性
P2	Human Approval + Application Tracker	需要长期使用时	形成求职 workflow 闭环
P3	Browser Agent	网站自动化确有需求后	成本高、易受登录/反爬/页面变化影响
P3	Web UI	CLI 逻辑稳定后	避免前端拖慢核心 Agent 开发

附录 A. 推荐目录结构

```

src/jobintel/
├── config.py
├── models.py
├── providers/
│   ├── base.py
│   ├── openai_provider.py
│   └── anthropic_provider.py
├── agent/
│   ├── core.py
│   ├── prompts.py
│   ├── tools.py
│   ├── guardrail.py
│   └── scoring.py
├── mcp_server/
│   ├── server.py
│   ├── repository.py
│   ├── db.py
│   ├── seed.py
│   └── errors.py
├── services/
│   ├── jd_parser.py
│   ├── evidence_search.py
│   └── requirement_matcher.py
└── cli.py

tests/
├── unit/
├── tools/
├── agent/
└── guardrail/

eval/
├── cases/
├── runner.py
├── metrics.py
└── report.py

```

附录 B. 与 AI 协作的任务指令模板

模板 1：实现单模块

任务：实现《JobIntel Agent 设计规格》6.2 节 JobRequirement。

允许修改：src/jobintel/models.py, tests/unit/test_models.py

禁止修改：agent/core.py, providers/*

要求：

1. 使用 Pydantic。
2. importance 只允许 must/preferred/bonus。
3. category 按文档枚举。
4. 新增至少 6 个单元测试，覆盖合法值、非法枚举、缺字段。
5. 不新增第三方依赖。

先给出修改计划，再给出 patch。

模板 2：修复 Bug

根据以下测试失败修复代码。只修改导致失败的最小范围。

必须保持第 9 章 Evidence Guardrail 规则不变。

输出：

1. 根因；2. 修改文件；3. 修改内容；4. 为什么不会破坏其他契约；5. 应新增的回归测试。

模板 3：代码审查

请按本设计文档审查以下代码，重点检查：

- 是否绕过 Pydantic contract
- 是否把 deterministic rule 放进了 Prompt
- 是否存在 matched without evidence
- Tool 是否职责过大
- 是否出现不必要的新依赖/框架
- 是否具备可测试性

按 Critical / Major / Minor 输出问题，不要直接大规模重写。

附录 C. V1 Definition of Done

- 可用 seed 数据启动，并运行至少 3 种不同岗位分析。
- Agent 至少会自主调用 3 类 Read Tool 和 1 类 Write Tool。
- 每个 matched Requirement 都有至少一个合法 evidence_id。
- 伪造 evidence_id、无证据 matched、跨候选人证据均被 Guardrail 拦截。
- 总分由 scoring.py 计算，不由 LLM 直接决定。
- 至少 30 个 Evaluation Cases。
- README 展示架构、工具列表、核心 Schema、Guardrail 机制、Eval 指标和示例输出。
- 单元测试、Tool 测试、Agent 离线测试能够一条命令运行。
- 代码中不依赖真实招聘网站才能完成 Demo。
- 能够在面试中清楚解释：为什么用 Agent、为什么用 MCP、为什么需要 Guardrail、为什么不用 RAG、如何评测。

参考项目

基础参考仓库：aawais-ai/sales-research-agent (GitHub)。本设计保留其小型、可读、Provider-independent Agent Tool Loop 与 MCP/Provenance 思路，但将业务领域、Evidence 模型、评分与 Evaluation 重构为 JobIntel 专用实现。

文档结束。后续开发时建议将本文作为“项目契约”，任何较大修改先更新设计，再修改代码。